

在当前上下文系统中加入高并发异步持久化

Redis 热快照 + Lua 原子提交 + Redis Streams 削峰 + MongoDB 异步落库

适用阶段	先完成同步持久化第一版，再通过配置开关切换到 Stream 模式
当前项目	agent_service / Redis DB 4 / MongoDB / LangGraph
推荐队列	Redis Streams；当前无需再部署 RabbitMQ
学习目标	削峰、原子提交、幂等、顺序、重试、死信、背压和可观测性

先给结论

可以做，而且很适合作为第一版之后的高并发练习。Python 协程只能提高等待期间的并发利用率，不能减少 MongoDB 的写入次数。Redis Streams 可以把请求线程中的 MongoDB 写入移动到后台消费者，形成削峰。但这会把 Redis 从普通缓存升级为“尚未落库上下文的暂存真源”，因此必须同时学习持久化、幂等、保序、重试和背压。

先理解：它和秒杀架构哪里相似，哪里不同

比较项	秒杀业务	智能体上下文
高峰来源	活动时开始时瞬时流量	产品故障、版本发布或集中咨询
Redis 作用	库存预扣、资格判断	热上下文提交和快速读取
队列作用	削峰后异步创建订单	削峰后异步持久化 MongoDB
核心顺序	同一商品库存顺序	同一 userId + sessionId 的 revision 顺序
重复处理	防止重复下单	requestId 幂等，防止重复增加 revision
失败代价	超卖或少卖	上下文丢失、旧上下文回答或版本错乱
消息体积	通常较小	可能包含摘要、工具、RAG 和近期轮次，明显更大

最大的相同点是“前台只做快速原子操作，把慢数据库写入交给消费者”；最大的不同点是上下文必须按会话严格保序，而且 Redis 中可能存在尚未写入 MongoDB 的唯一最新版本。

最终运行流程

```
LangGraph 正常完成
-> output_validate
-> persist_context
    -> Redis Lua 校验 requestId 与 expectedRevision
    -> XADD 不可变持久化事件
    -> SET 最新热快照 + revision
-> Redis 返回 COMMITTED
-> SSE done (表示事件已被可靠接收, 不表示 MongoDB 已完成)
```

```
后台消费者
-> XREADGROUP 按分片读取
-> requestId 幂等写 context_turns
-> revision 有序更新 context_sessions
-> 成功后 XACK
-> 失败不 ACK, 稍后重试 / XAUTOCLAIM / 死信
```

语义变化

同步版的 done 表示 MongoDB 已提交；Stream 版的 done 表示 Redis 快照与持久化事件已原子接收。MongoDB 允许短暂落后。这个语义必须写进接口文档、健康检查和监控。

第 1 步：先用配置开关保留同步版

不要直接删除已经学会的 MongoDB-first 实现。先让两种模式共存，便于对比延迟、故障和数据结果。

修改配置 agent_service/.env.example

```
# sync: 当前第一版，回答完成后同步保存 MongoDB，再更新 Redis。
# stream: 高并发学习版，Redis 原子提交后由 Stream 消费者异步保存 MongoDB。
AGENT_CONTEXT_PERSIST_MODE=sync

# 固定分片数。相同会话总是进入相同 Stream，保证该会话的事件有序。
AGENT_CONTEXT_STREAM_SHARDS=8
AGENT_CONTEXT_STREAM_GROUP=mongo_persisters
AGENT_CONTEXT_STREAM_BLOCK_MS=5000
AGENT_CONTEXT_STREAM_RETRY_IDLE_MS=60000
AGENT_CONTEXT_STREAM_MAX_DELIVERIES=5

# 示例阈值，不是生产环境固定答案；压测后调整。
AGENT_CONTEXT_QUEUE_DEGRADED_LAG=1000
AGENT_CONTEXT_QUEUE_REJECT_LAG=10000

# 快照七天滑动过期；requestId 幂等标记保留三十天。
AGENT_CONTEXT_REDIS_TTL_SECONDS=604800
AGENT_CONTEXT_IDEMPOTENCY_TTL_SECONDS=2592000
```

修改文件 agent_service/src/agent_service/config.py

```
from typing import Literal

# 保留同步实现，只有显式配置 stream 才启用异步持久化。
context_persist_mode: Literal["sync", "stream"] = "sync"
```

```

context_stream_shards: int = Field(default=8, ge=1, le=64)
context_stream_group: str = "mongo_persisters"
context_stream_block_ms: int = Field(default=5000, ge=100)
context_stream_retry_idle_ms: int = Field(default=60000, ge=1000)
context_stream_max_deliveries: int = Field(default=5, ge=1)
context_queue_degraded_lag: int = Field(default=1000, ge=0)
context_queue_reject_lag: int = Field(default=10000, ge=1)
context_idempotency_ttl_seconds: int = Field(default=2592000, ge=60)

# 配置校验: 拒绝阈值必须大于降级阈值。
if self.context_queue_reject_lag <= self.context_queue_degraded_lag:
    raise ValueError(
        "AGENT_CONTEXT_QUEUE_REJECT_LAG 必须大于 "
        "AGENT_CONTEXT_QUEUE_DEGRADED_LAG"
    )

```

第 2 步: 定义 Redis Key 与会话分片

不能使用 Python 内置 `hash()` 做分片, 因为不同进程启动后结果可能变化。使用稳定哈希, 让同一会话永远进入同一条 Stream。

新增文件 `agent_service/src/agent_service/services/context_stream_keys.py`

```

"""上下文 Stream 的 Key 规则与稳定分片。"""

import hashlib

PREFIX = "xc:agent:context:v2"

def session_tag(user_id: int, session_id: int) -> str:
    """返回稳定的会话标识。"""
    return f"{user_id}:{session_id}"

def shard_for_session(user_id: int, session_id: int, shard_count: int) -> int:
    """把同一会话稳定映射到同一个 Stream 分片。"""
    raw = session_tag(user_id, session_id).encode("utf-8")
    digest = hashlib.blake2b(raw, digest_size=8).digest()
    number = int.from_bytes(digest, byteorder="big", signed=False)
    return number % shard_count

def snapshot_key(user_id: int, session_id: int) -> str:
    return f"{PREFIX}:snapshot:{user_id}:{session_id}"

def revision_key(user_id: int, session_id: int) -> str:
    return f"{PREFIX}:revision:{user_id}:{session_id}"

def request_key(request_id: int) -> str:
    # requestId 幂等标记, 防止同一个请求重复产生 Stream 消息。
    return f"{PREFIX}:request:{request_id}"

def stream_key(shard: int) -> str:
    return f"{PREFIX}:persist:{shard}"

def dead_letter_key(shard: int) -> str:

```

```
return f"{PREFIX}:dead:{shard}"
```

当前 Redis 是单机

上述 Lua 会同时访问会话 Key 和分片 Stream，在当前单机 Redis 中没有问题。以后切换 Redis Cluster 时，多 Key 脚本要求 Key 位于同一 hash slot，需要重新设计 hash tag、改成每会话 Stream，或改用 Kafka/RabbitMQ。不要直接照搬到 Cluster。

第 3 步：定义不可变的持久化事件

消费者不能等到稍后再读取“当前 Redis 快照”来猜测当时发生了什么，因为 Redis 可能已经前进到更高 revision。Stream 事件必须携带本次提交所需的不可变数据。

新增文件 `agent_service/src/agent_service/schemas/context_event.py`

```
"""异步持久化事件模型。"""

from datetime import UTC, datetime

from pydantic import Field

from agent_service.schemas.context import (
    ContextEntry,
    ContextModel,
    ContextSnapshot,
)

class ContextPersistenceEvent(ContextModel):
    """Redis 已接受、等待 MongoDB 持久化的一次上下文提交。"""

    # 事件结构版本，后续字段变化时用于兼容旧消息。
    event_version: int = 1

    request_id: int = Field(gt=0)
    user_id: int = Field(gt=0)
    session_id: int = Field(gt=0)

    # expected_revision 是提交前版本；revision 是提交成功后的版本。
    expected_revision: int = Field(ge=0)
    revision: int = Field(gt=0)

    # 单轮记录用于幂等写 context_turns。
    turn: ContextEntry

    # 当前快照用于更新 context_sessions 投影视图。
    # 第一版为便于学习直接携带完整快照；后续可压缩或改为事件增量。
    snapshot: ContextSnapshot

    committed_at: datetime = Field(
        default_factory=lambda: datetime.now(UTC)
    )
```

完整快照会增加 Stream 内存占用，但能降低第一版消费者重建难度。完成正确性验证后，再评估只投递 turn + summary/facts 变化量。

第 4 步：用 Lua 提交快照与 Stream 事件

Lua 的目的不是让 Redis 代替所有事务，而是保证其他请求看不到“只更新快照、没有消息”或“只有消息、快照没更新”的中间状态。脚本必须短小，不能做压缩或复杂 JSON 运算。

新增文件 `agent_service/src/agent_service/scripts/commit_context.lua`

```
-- KEYS[1]: snapshot key
-- KEYS[2]: revision key
-- KEYS[3]: requestId 幂等 key
-- KEYS[4]: 目标 Stream key
--
-- ARGV[1]: expected revision
-- ARGV[2]: new revision
-- ARGV[3]: snapshot JSON
-- ARGV[4]: event JSON
-- ARGV[5]: requestId
-- ARGV[6]: snapshot/revision TTL 秒数
-- ARGV[7]: requestId 幂等 TTL 秒数

local duplicated_revision = redis.call('GET', KEYS[3])
if duplicated_revision then
    return {'DUPLICATE', duplicated_revision, ''}
end

local current_revision = tonumber(redis.call('GET', KEYS[2]) or '0')
local expected_revision = tonumber(ARGV[1])
if current_revision ~= expected_revision then
    return {'REVISION_CONFLICT', tostring(current_revision), ''}
end

-- 先执行最可能因 Key 类型错误而失败的 XADD。
-- Redis Lua 保证脚本执行期间不会被其他命令穿插，但运行时错误不会自动回滚
-- 已经执行的写操作，所以部署时必须保证 Stream Key 只用于 Stream。
local stream_id = redis.call(
    'XADD', KEYS[4], '*',
    'payload', ARGV[4],
    'requestId', ARGV[5],
    'revision', ARGV[2]
)

-- XADD 成功后更新热快照、revision 和幂等标记。
redis.call('SET', KEYS[1], ARGV[3], 'EX', ARGV[6])
redis.call('SET', KEYS[2], ARGV[2], 'EX', ARGV[6])
redis.call('SET', KEYS[3], ARGV[5], 'EX', ARGV[7])

return {'COMMITTED', ARGV[2], stream_id}
```

不要在 XADD 中立刻 MAXLEN 裁剪

如果生产者按长度裁剪 Stream，可能删除尚未被消费者持久化的消息。第一版先保留消息并监控内存，后续由维护任务结合 Pending 和消费进度安全使用 XTRIM MINID。

第 5 步：实现 Redis Stream 提交器

新增文件 `agent_service/src/agent_service/services/context_stream_writer.py`

```
"""把上下文原子提交到 Redis 热快照与持久化 Stream。"""

from pathlib import Path

from redis.asyncio import Redis
from redis.exceptions import RedisError

from agent_service.config import Settings
from agent_service.core.exceptions import (
    ContextQueueUnavailableError,
    ContextRevisionConflictError,
)
from agent_service.schemas.context import ContextEntry, ContextSnapshot
from agent_service.schemas.context_event import ContextPersistenceEvent
from agent_service.services.context_stream_keys import (
    request_key,
    revision_key,
    shard_for_session,
    snapshot_key,
    stream_key,
)

class RedisStreamContextWriter:
    """高并发模式下的前台提交器。"""

    def __init__(self, redis: Redis, settings: Settings) -> None:
        self._redis = redis
        self._settings = settings
        script_path = (
            Path(__file__).parents[1] / "scripts" / "commit_context.lua"
        )
        self._commit_script = script_path.read_text(encoding="utf-8")

    async def commit(
        self,
        *,
        snapshot: ContextSnapshot,
        turn: ContextEntry,
        expected_revision: int,
    ) -> ContextSnapshot:
        """提交成功即表示 Redis 已接受快照和持久化事件。"""

        new_revision = expected_revision + 1
        saved = snapshot.model_copy(
            update={"revision": new_revision},
            deep=True,
        )
        event = ContextPersistenceEvent(
            request_id=turn.request_id,
            user_id=saved.user_id,
            session_id=saved.session_id,
            expected_revision=expected_revision,
            revision=new_revision,
            turn=turn,
            snapshot=saved,
        )
        shard = shard_for_session(
            saved.user_id,
            saved.session_id,
            self._settings.context_stream_shards,
```

```

    )

    try:
        result = await self._redis.eval(
            self._commit_script,
            4,
            snapshot_key(saved.user_id, saved.session_id),
            revision_key(saved.user_id, saved.session_id),
            request_key(turn.request_id),
            stream_key(shard),
            expected_revision,
            new_revision,
            saved.model_dump_json(by_alias=True),
            event.model_dump_json(by_alias=True),
            turn.request_id,
            self._settings.context_redis_ttl_seconds,
            self._settings.context_idempotency_ttl_seconds,
        )
    except RedisError as exc:
        # Stream 模式下 Redis 是提交入口，不能像普通缓存一样忽略失败。
        raise ContextQueueUnavailableError(
            "Redis 上下文提交失败"
        ) from exc

    status = self._decode(result[0])
    actual_revision = int(self._decode(result[1]))

    if status == "REVISION_CONFLICT":
        raise ContextRevisionConflictError(
            f"expected={expected_revision}, actual={actual_revision}"
        )

    # DUPLICATE 表示相同 requestId 已成功提交，不再产生第二条消息。
    if status == "DUPLICATE":
        return saved.model_copy(
            update={"revision": actual_revision},
            deep=True,
        )

    return saved

    @staticmethod
    def _decode(value: bytes | str) -> str:
        return value.decode("utf-8") if isinstance(value, bytes) else value

```

生产代码可在启动时 SCRIPT LOAD，并优先 EVALSHA；遇到 NOSCRIPT 再回退 EVAL。示例先使用 eval，便于理解完整调用参数。

第 6 步：让 LangGraph 保存节点只做快速提交

修改文件 agent_service/src/agent_service/graph/workflow.py

```

async def persist_context(
    state: AgentState,
    writer: StreamWriter,
) -> dict[str, object]:
    """保存完整轮次；stream 模式不在请求协程里写 MongoDB。"""

    writer(
        {
            "event": "status",
            "payload": {
                "stage": "context",

```

```

        "message": "正在提交会话上下文",
    },
}
)

turn = ContextEntry(
    request_id=state["request_id"],
    user_text=state["message"],
    assistant_text=state["final_answer"],
    status="completed",
    tool_contexts=state.get("tool_contexts", []),
    rag_contexts=state.get("rag_contexts", []),
)
next_snapshot = context_manager.append_completed_turn(
    snapshot=state["context_snapshot"],
    turn=turn,
)

if settings.context_persist_mode == "sync":
    # 保留第一版: MongoDB 同步保存, Redis 作为缓存。
    saved = await context_repository.save(
        next_snapshot,
        expected_revision=state["context_revision"],
    )
else:
    # 高并发版: Redis 快照 + Stream 事件原子提交。
    saved = await context_stream_writer.commit(
        snapshot=next_snapshot,
        turn=turn,
        expected_revision=state["context_revision"],
    )

return {
    "context_snapshot": saved,
    "context_revision": saved.revision,
}

```

done 的发送顺序

路由仍然必须等 `persist_context` 返回后才发送 `done`。Stream 模式只是把等待点从 MongoDB 改成 Redis 原子提交，并不是完全不等待保存。

第 7 步：创建独立 MongoDB 消费者

消费者应作为独立进程运行，不要和 FastAPI 请求进程共用阻塞式 XREADGROUP 连接。一个分片只安排一个有序工作协程；不同分片并行，从而同时获得会话内保序和会话间并发。

新增文件 `agent_service/src/agent_service/workers/context_persistence_consumer.py`

```

"""Redis Streams -> MongoDB 的异步持久化消费者。"""

import asyncio
import logging

from redis.asyncio import Redis
from redis.exceptions import ResponseError

from agent_service.schemas.context_event import ContextPersistenceEvent
from agent_service.services.context_stream_keys import stream_key

```



```

logger = logging.getLogger(__name__)

class ContextPersistenceConsumer:
    def __init__(self, redis: Redis, mongo_repository, settings) -> None:
        self._redis = redis
        self._mongo_repository = mongo_repository
        self._settings = settings

    async def ensure_groups(self) -> None:
        """为每条分片 Stream 创建消费者组。"""
        for shard in range(self._settings.context_stream_shards):
            try:
                await self._redis.xgroup_create(
                    name=stream_key(shard),
                    groupname=self._settings.context_stream_group,
                    id="0-0",
                    mkstream=True,
                )
            except ResponseError as exc:
                if "BUSYGROUP" not in str(exc):
                    raise

    async def run(self) -> None:
        """每个分片启动一个顺序工作协程。"""
        await self.ensure_groups()
        async with asyncio.TaskGroup() as group:
            for shard in range(self._settings.context_stream_shards):
                group.create_task(self._run_shard(shard))

    async def _run_shard(self, shard: int) -> None:
        stream = stream_key(shard)
        group = self._settings.context_stream_group

        # 固定消费者名便于进程重启后继续读取自己的 Pending 消息。
        consumer = f"mongo-persister-{shard}"

        while True:
            # 先读取当前消费者尚未 ACK 的消息，避免失败消息被跳过。
            response = await self._redis.xreadgroup(
                groupname=group,
                consumername=consumer,
                streams={stream: "0"},
                count=1,
            )

            if not response:
                # 没有自己的 Pending 消息，再读取一条从未投递的新消息。
                response = await self._redis.xreadgroup(
                    groupname=group,
                    consumername=consumer,
                    streams={stream: ">"},
                    count=1,
                    block=self._settings.context_stream_block_ms,
                )

            if not response:
                continue

            _, messages = response[0]
            message_id, fields = messages[0]

            try:
                event = ContextPersistenceEvent.model_validate_json(
                    fields[b"payload"]
                )
            except:
                pass
            await self._mongo_repository.persist_event(event)

```

```

        # 只有两个 MongoDB 投影都完成后才能 ACK。
        await self._redis.xack(stream, group, message_id)
    except Exception:
        logger.exception(
            "上下文持久化失败, stream=%s messageId=%s",
            stream,
            message_id,
        )
    # 不 ACK, 保留在 Pending 中; 短暂退避后重新处理同一条。
    await asyncio.sleep(1)

```

第 8 步: MongoDB 消费必须幂等、可重放

Redis Consumer Group 是至少一次投递。消费者可能在 MongoDB 写成功后、XACK 前崩溃, 所以同一事件一定可能再次执行。requestId 唯一索引和 revision 条件更新不是可选项。

修改文件 `agent_service/src/agent_service/services/context_repository.py`

```

async def ensure_indexes(self) -> None:
    """消费者启动前创建幂等和查询索引。"""
    await self._turns.create_index(
        [("requestId", 1)],
        unique=True,
        name="uk_context_turn_request",
    )
    await self._sessions.create_index(
        [("userId", 1), ("sessionId", 1)],
        unique=True,
        name="uk_context_session",
    )

async def persist_event(self, event: ContextPersistenceEvent) -> None:
    """可重复执行地把一个 Stream 事件投影到 MongoDB。"""

    turn_document = event.turn.model_dump(mode="python", by_alias=True)
    turn_document.update(
        {
            "userId": event.user_id,
            "sessionId": event.session_id,
            "revision": event.revision,
        }
    )

    # 1. 先保存不可变轮次事件。
    # 重复消费相同 requestId 时不会插入第二条。
    await self._turns.update_one(
        {"requestId": event.request_id},
        {"$setOnInsert": turn_document},
        upsert=True,
    )

    identity = {
        "userId": event.user_id,
        "sessionId": event.session_id,
    }
    snapshot_document = event.snapshot.model_dump(
        mode="python",
        by_alias=True,
    )

```

```

# 2. 读取当前投影版本。
current = await self._sessions.find_one(
    identity,
    projection={"revision": 1},
)

if current is None:
    try:
        await self._sessions.insert_one(snapshot_document)
        return
    except DuplicateKeyError:
        # 另一个重试刚刚创建成功，继续走下面的版本判断。
        current = await self._sessions.find_one(
            identity,
            projection={"revision": 1},
        )

current_revision = int(current.get("revision", 0))

if current_revision >= event.revision:
    # 相同或更高版本已经存在，本次重复/旧事件可安全视为成功。
    return

if current_revision + 1 != event.revision:
    # 说明同一会话事件乱序。不要跳过缺失版本，留在 Pending 重试。
    raise ContextEventOutOfOrderError(
        f"current={current_revision}, incoming={event.revision}"
    )

result = await self._sessions.replace_one(
    {**identity, "revision": current_revision},
    snapshot_document,
)
if result.matched_count == 0:
    # 并发投影冲突，保留消息稍后重试。
    raise ContextRevisionConflictError("MongoDB 投影版本已变化")

```

当前 MongoDB 是单机容器，不能依赖多文档事务。先幂等写 context_turns，再更新

context_sessions；如果中间崩溃，消息未 ACK，重放时会补齐快照。以后改成副本集后可以再评估事务。

第 9 步：为什么需要分片保序

如果一个 Stream Consumer Group 中直接启动很多消费者，revision=4 和 revision=5 可能被两个消费者同时处理。数据库会拒绝乱序，但会产生大量重试。固定分片能让相同会话保持顺序，不同会话并行。

```

session A -> shard 2 -> revision 1, 2, 3 顺序处理
session B -> shard 5 -> revision 1, 2 顺序处理
session C -> shard 2 -> 与 A 共用工作协程，但仍按 Stream 顺序

```

8 个 shard = 8 个 MongoDB 持久化工作协程
增加并发时扩充分片数，而不是给同一分片随意增加并行消费者。

分片数需要谨慎

分片数参与路由，运行中直接从 8 改成 16 会让同一会话进入另一条 Stream。正式切换时需要版本化 Stream 前缀、排空旧分片或执行迁移，不能只改环境变量重启。

第 10 步：处理 Pending、消费者崩溃和死信

消费者读到消息后会进入 Pending Entries List，只有 XACK 才算完成。如果消费者永久退出，需要另一个恢复任务用 XAUTOCLAIM 接管长时间空闲的 Pending 消息。超过重试次数后转移到死信 Stream，并报警。

新增逻辑 context_persistence_consumer.py / reclaim_stale_messages

```

async def reclaim_stale_messages(self, shard: int) -> None:
    """接管崩溃消费者遗留的 Pending 消息。"""
    stream = stream_key(shard)
    group = self._settings.context_stream_group
    consumer = f"mongo-persister-{shard}"
    cursor = "0-0"

    while True:
        # redis-py 版本不同，xautoclaim 返回结构可能略有差异：
        # 实现时应按当前 uv.lock 中的 redis 版本编写解析测试。
        result = await self._redis.xautoclaim(
            name=stream,
            groupname=group,
            consumername=consumer,
            min_idle_time=self._settings.context_stream_retry_idle_ms,
            start_id=cursor,
            count=20,
        )

        cursor, messages = result[0], result[1]
        if not messages:
            break

        # 被接管的消息已经属于当前消费者；主循环读取 Pending 后重试。
        if cursor in ("0-0", b"0-0"):
            break

    # 死信处理原则（伪代码）
    if delivery_count >= max_deliveries:
        XADD dead_stream payload=<原始事件> reason=<稳定错误码>
        XACK original_stream group message_id
        记录告警并保留 requestId、sessionId、revision

    # 不要先 ACK 再写死信，否则写死信失败时消息永久丢失。

```

第 11 步：增加积压监控与背压

消息队列能削峰，但不能无限吸收流量。如果产品故障持续数小时而 MongoDB 消费速度低于生产速度，Redis 内存最终仍会耗尽。生产者必须能根据 lag 进入降级或拒绝状态。

指标	含义	建议动作
stream lag	消费者组尚未投递的消息	持续上升时扩容消费者或限流
pending count	已经投递但未 ACK	上升通常说明 MongoDB 失败或消费者卡住
oldest pending	最老未完成消息年龄	超过目标时间触发 XAUTOCLAIM/告警
DLQ length	无法自动恢复的事件	任何增长都需要人工检查
Mongo write latency	消费者单条/批量落库耗时	用于判断扩容和索引问题
Redis memory	快照、Stream 和 Pending 占用	接近上限前必须限流，禁止依赖淘汰

新增服务 `agent_service/src/agent_service/services/context_queue_health.py`

```
class ContextQueueHealth:
    """由后台监控任务周期更新，不要每个请求都执行 XINFO。"""

    def __init__(self) -> None:
        self.max_lag = 0
        self.pending = 0

    def ensure_writable(self, settings: Settings) -> None:
        if self.max_lag >= settings.context_queue_reject_lag:
            raise ContextQueueOverloadedError(
                "上下文持久化积压过高，请稍后重试"
            )

        # degraded 阶段可以继续接收，但健康检查返回 DEGRADED，
        # 并触发告警或降低模型并发。
```

不要使用 `volatile-lru` 淘汰未落库数据

Stream 模式下 Redis 不再只是可丢弃缓存。需要检查 `maxmemory-policy`，避免快照或 Stream 消息在 MongoDB 持久化前被淘汰。队列接近内存上限时应背压，而不是指望 Redis 自动淘汰。

第 12 步：把消费者做成独立 uv 入口

修改文件 `agent_service/pyproject.toml`

```
[project.scripts]
agent-service = "agent_service.main:run"
agent-context-consumer = "agent_service.consumer_main:run"
```

新增文件 `agent_service/src/agent_service/consumer_main.py`

```
"""MongoDB 上下文持久化消费者入口。"""

import asyncio

from pymongo import AsyncMongoClient
from redis.asyncio import Redis

from agent_service.config import get_settings
from agent_service.services.context_repository import MongoContextProjectionRepository
from agent_service.workers.context_persistence_consumer import (
    ContextPersistenceConsumer,
)
```

```

async def main() -> None:
    settings = get_settings()
    redis = Redis(
        host=settings.redis_host,
        port=settings.redis_port,
        password=(
            settings.redis_password.get_secret_value()
            if settings.redis_password
            else None
        ),
        db=settings.redis_database,
        decode_responses=False,
    )
    mongo = AsyncMongoClient(
        settings.mongodb_uri.get_secret_value(),
        serverSelectionTimeoutMS=settings.mongodb_server_selection_timeout_ms,
    )

    repository = MongoContextProjectionRepository(
        mongo[settings.mongodb_database]
    )
    await repository.ensure_indexes()

    consumer = ContextPersistenceConsumer(redis, repository, settings)
    try:
        await consumer.run()
    finally:
        await redis.aclose()
        await mongo.close()

def run() -> None:
    asyncio.run(main())

```

开发时需要分别启动 API 和消费者：

```

# 终端 1: FastAPI / LangGraph 请求服务
uv run agent-service

# 终端 2: Redis Streams -> MongoDB 消费者
uv run agent-context-consumer

```

第 13 步：理解 Redis 持久化的新责任

在同步版中 Redis 丢失只会造成缓存未命中；在 Stream 版中 Redis 可能保存尚未落 MongoDB 的最新上下文和消息。Redis 宕机后的数据损失窗口必须成为明确的业务选择。

```

# 在虚拟机检查当前 Redis 持久化配置：不要把密码写进文档或脚本。
redis-cli -h 192.168.100.128 -p 6379 -a <Redis 密码> \
    CONFIG GET appendonly appendfsync maxmemory-policy

# AOF everysec: 性能与可靠性的常见折中；极端故障可能损失约一秒写入。
# AOF always: 每批写入都 fsync，更可靠，但会增加延迟和磁盘压力。

```

- 个人学习环境可以先使用 AOF everysec，并主动做故障实验。
- 正式智能客服不应把单节点 Redis 当成无损消息队列，需要副本、磁盘监控和备份。
- 如果 Redis 不可用且 MongoDB 可能落后，不能直接静默读取 MongoDB 继续回答；应返回 CONTEXT_QUEUE_UNAVAILABLE 或进入经过设计的降级流程。

第 14 步：按影子模式迁移，不一次性切换

1. 先完成 sync 第一版和已有测试，建立正确性基线。
2. 开启 shadow 模式：正常请求仍同步写 MongoDB，同时投递 Stream 到测试集合，对比结果。
3. 验证 requestId、revision、摘要、工具和 RAG 字段完全一致。
4. 进行消费者崩溃、MongoDB 停机、Redis 重启和积压恢复测试。
5. 确认 lag、Pending、DLQ 和 Redis 内存监控有效后，再切换 stream。
6. 保留快速回退到 sync 的配置，但切换前必须先排空 Stream，避免双重写入。

必须覆盖的测试

- Lua 正常提交：快照、revision、request 幂等标记和 Stream 事件同时可见。
- 重复 requestId：返回 DUPLICATE，不生成第二条 Stream 消息。
- 错误 expectedRevision：返回 REVISION_CONFLICT，快照和 Stream 都不变化。
- 消费者在 MongoDB 写成功、XACK 前崩溃：重启后重复消费但不产生重复数据。
- MongoDB 停机：消息留在 Pending，Redis 快照仍能服务后续会话，恢复后按序落库。
- 同一会话连续 revision 1/2/3：MongoDB 最终顺序一致；不同会话可以并行。
- 消费者永久退出：XAUTOCLAIM 能接管旧 Pending；超过次数进入死信。
- 积压超过拒绝阈值：新提交返回 CONTEXT_QUEUE_OVERLOADED，不继续吃满 Redis。
- Redis 重启：验证 AOF 恢复的快照、revision、Stream 和 Pending 行为。
- Stream 清理：不会裁剪尚未 ACK 的消息。

压测时看什么，而不只是看 QPS

观察项	问题
API P95/P99	Redis 原子提交是否稳定，是否被 AOF 或大消息拖慢
生产/消费速率	每秒新增事件是否长期高于 MongoDB 消费能力
lag 恢复时间	流量高峰结束后多久能追平 MongoDB
单条消息大小	完整 snapshot 是否导致 Redis 内存和网络压力
同会话顺序	高并发下是否出现 ContextEventOutOfOrderError

第一阶段不要一起增加的内容

先完成 Redis Streams -> MongoDB 的正常完成轮次，不要同时实现取消轮次收尾、RAG 新架构、工具并行、Redis Cluster 或 Kafka。取消一致性已经单独记录，等基础异步持久化稳定后再合并，否则故障路径会成倍增加。

参考接口

- Redis Streams、Consumer Groups、XREADGROUP、XACK、XAUTOCLAIM：
<https://redis.io/docs/latest/develop/data-types/streams/>
- Redis Lua 原子执行与 EVAL/EVALSHA：
<https://redis.io/docs/latest/develop/interact/programmability/eval-intro/>
- Redis AOF 与 RDB 持久化权衡：
https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
- PyMongo Bulk Write（后续批量优化）：
<https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/bulk-write/>
- MongoDB 唯一索引与复合索引：<https://www.mongodb.com/docs/manual/core/index-unique/>

建议的学习终点

不是“接口返回更快”就算完成，而是高峰期间 API 能快速提交、MongoDB 可以短暂落后、消费者恢复后数据最终一致、同一会话严格保序、重复消息无副作用、积压过高能够主动背压。